Discrete Ordinates Method (DOM)

Full Guide vs. DOM_v2 Implementation

EE 645 - Radiative Transfer in the Environment

This document summarises Ranga's 11-step DOM Full Guide and explains how the DOM_v2 Python implementation maps those 11 steps into 6 modular files. Structural differences, consolidations, and extensions are highlighted.

Part 1 - The DOM Full Guide (Ranga's 11 Steps)

The course reference uses an 11-step algorithm to solve the canopy radiative transfer equation with the Discrete Ordinates Method. The radiation field is decomposed into an *uncollided* component (direct + diffuse sky that has never hit a leaf) and a *collided* component (photons that have scattered at least once). This split avoids treating the forward-peaked beam source inside the iteration loop.

Step 1 - Angular Discretization

- N Gauss-Legendre nodes mu_n and weights w_mu on [-1, +1]. N must be even; first N/2 nodes are downward (mu < 0), last N/2 are upward (mu > 0).
- M uniform azimuth angles phi_m with weight w_phi = 2*pi/M.
- Pre-compute phase-function tables: G_qq[n,m,i,j] for Gauss-to-Gauss scattering and G_sol[i,j] for solar-beam-to-Gauss scattering. These are the most expensive setup operations and are computed only once per spectral band.

Step 2 - Spatial Discretization

- Divide canopy (LAI) into K equal layers; cell thickness dL = LAI/K.
- Leaf projection coefficient G = 0.5 for a uniform leaf-angle distribution.
- Per-cell optical depth: tau = G * dL.

Step 3a - Downward Sweep - Direct Solar Beam (Uncollided)

- Beer-Lambert attenuation for the collimated solar beam:
- I0_dir[k+1] = I0_dir[k] * exp(-G*dL / |mu_solar|)
- Starting condition at canopy top: I0_dir[1] = cos(theta_solar) * E0.

Step 3b - Downward Sweep - Diffuse Sky (Uncollided)

- For each downward direction (n, m) independently:
- I0[n,m,k+1] = I0[n,m,k] * exp(-G*dL / |mu_n|)
- Upper BC: isotropic sky radiance L_sky for all downward directions.

Step 4 - Lower Boundary Condition - Uncollided

- Lambertian ground reflection combines direct and diffuse downward fluxes:
- I0_up_gnd[n,m] = (rho_g / pi) * (Fd_dir[K+1] + Fd_dif[K+1])
- rho_g is the diffuse ground reflectance; the factor 1/pi ensures energy conservation.

Step 5 - Upward Sweep - Uncollided

- From the ground BC, sweep upward using Beer-Lambert:
- I0[n,m,k] = I0[n,m,k+1] * exp(-G*dL / |mu_n|) (upward directions)
- Completes the full uncollided field I0 across the canopy.

Step 6 - First-Collision Source Q = Q1 + Q2 + Q3

- Q seeds the collided iteration. Three contributions evaluated at cell-centre depths:
- Q1[i,j,k] = (omega_L/pi) * G_sol[i,j] * I0_dir_c[k] (solar beam -> direction (i,j))
- Q2[i,j,k] = (omega_L/pi) * SUM w_n*w_phi * G_qq[n,m,i,j] * I0_down_c[n,m,k] (sky diffuse -> (i,j))
- Q3[i,j,k] = (omega_L/pi) * SUM w_n*w_phi * G_qq[n,m,i,j] * I0_up_c[n,m,k] (ground-reflected -> (i,j))

Step 7 - Diamond-Difference Coefficients

- Using SIGNED mu (negative=downward, positive=upward) and f[n] = G*dL / |mu_n|:
- Downward (mu < 0): a = (1+(1-alpha)*f)/(1-alpha*f), b = f/(G*(1-alpha*f))
- Upward (mu > 0): c = (1-(1-alpha)*f)/(1+alpha*f), d = f/(G*(1+alpha*f))
- alpha=0 is the standard DD scheme; alpha=0.5 gives the step (upwind) scheme.

Step 8 - Collided Downward Sweep

- Using total source J = Q + S (S=0 on first pass):
- IC[k+1] = a*IC[k] - b*J[k] for downward directions
- Upper BC: IC = 0 (no incoming collided radiation at canopy top).
- Collided ground BC recomputed from THIS sweep:
- IC_gnd = (rho_g/pi) * SUM w_n*w_phi*|mu_n| * IC_down[n,m,K+1]

Step 9 - Collided Upward Sweep

- IC[k] = c*IC[k+1] + d*J[k] for upward directions
- Lower BC: IC_gnd from the downward sweep of THIS iteration (critical for consistency).
- After sweep, update scatter source S for next iteration:
- S[i,j,k] = (omega_L/pi) * SUM w_n*w_phi * G_qq[n,m,i,j] * IC_c[n,m,k]

Step 10 - Iteration Loop - Dual Convergence

- Repeat Steps 8-9 until BOTH criteria are satisfied:
- (A) Max relative change in IC at all boundary nodes < epsilon
- (B) Max relative change in scalar irradiance profile < epsilon
- Scalar irradiance (no cosine weight): SI[k] = SUM w_n*w_phi*(I0_c+IC_c) + I0_dir_c
- Dual convergence prevents premature termination from a single metric.

Step 11 - Energy Balance Diagnostics

- Assemble flux profiles and verify conservation:
- F_ref + A_leaves + T_ground = F_in (should be 1.0 within machine precision)
- A_leaves = G*(1-omega_L)*dL * SUM_k SI[k] (scalar irradiance method, preferred)
- Alternative: A = flux divergence sum over all layers.

Part 2 - DOM_v2 Implementation (6 Steps)

DOM_v2 reorganises the 11-step guide into 6 Python modules, each with a single clear responsibility. The underlying physics is identical; the difference is only how the algorithm is partitioned into files.

step1_quadrature.py - Angular Discretization

Builds polar and azimuthal quadrature on the full sphere. Four methods are supported: gauss_legendre (default), double_gauss (separate GL sets for each hemisphere), gauss_lobatto, and uniform. Returns mu, w_mu, phi, w_phi. Spatial parameters (K, dL) are handled in config.py, not here.

step2_phase.py - Phase Function Tables

Computes the bi-Lambertian scattering phase function Gamma(i->j). Two lookup tables are built: G_qq of shape (N+1, M+1, N+1, M+1) for Gauss-to-Gauss scattering, and G_sol of shape (N+1, M+1) for the solar-beam seeding. Both are computed once per spectral band and reused throughout all iterations.

step3_uncollided.py - Uncollided Field + First-Collision Source

Consolidates guide Steps 3a, 3b, 4, 5, and 6 into a single function. The complete uncollided solve (downward direct, downward diffuse, Lambertian ground BC, upward sweep) is performed sequentially, after which Q = Q1 + Q2 + Q3 is assembled. Returns I0, I0_dir, flux profiles Fd_dir/Fd_dif/Fu_unc, and Q.

step4_collided.py - Collided Field - Iterative DD Solver

Implements guide Steps 7 through 10. DD coefficients are precomputed once. The iteration loop: compute J=Q+S, downward DD sweep (upper BC=0), recompute collided ground BC from THIS sweep, upward DD sweep, update scatter source S, check dual convergence. Returns converged IC, Fd_col, Fu_col.

step5_energy.py - Energy Balance Diagnostics

Implements guide Step 11. Assembles total flux profiles Fd_tot and Fu_tot, and computes scalar irradiance SI. Verifies leaf absorption via the scalar irradiance method and checks F_ref + A_leaves + T_ground = F_in.

step6_brf.py - BRF / HDRF at Arbitrary View Directions [Extension]

Not present in the original 11-step guide. Computes the Bi-directional Reflectance Factor (or HDRF) at any user-specified view angle without re-running the full iteration. Builds the view-direction source J_v from the converged source field, runs one additional upward DD sweep along the view ray, and returns BRF = pi * I_v(top) / (cos(theta_solar) * F_in).

Part 3 - Step-by-Step Mapping

The table below shows how each of Ranga's 11 guide steps maps onto the 6 DOM_v2 files.

Guide Step	Topic	DOM_v2 File
Step 1	Angular discretization + phase tables	step1_quadrature.py + step2_phase.py
Step 2	Spatial discretization (K, dL, G)	config.py / main.py
Steps 3a-3b	Uncollided downward sweeps (direct + diffuse)	step3_uncollided.py
Step 4	Lower BC - Lambertian ground (uncollided)	step3_uncollided.py
Step 5	Uncollided upward sweep	step3_uncollided.py
Step 6	First-collision source Q = Q1+Q2+Q3	step3_uncollided.py
Steps 7-9	DD coefficients + collided sweeps (down+up)	step4_collided.py
Step 10	Iteration loop with dual convergence	step4_collided.py
Step 11	Energy balance & scalar irradiance diagnostics	step5_energy.py
(Extension)	BRF/HDRF at arbitrary view angles	step6_brf.py

Table 1. Correspondence between Ranga's 11-step guide and the 6 DOM_v2 modules.

Part 4 - Key Differences and Design Choices

1. Consolidation of the uncollided solve

The guide spreads the uncollided field across four separate steps (3a, 3b, 4, 5) plus a dedicated first-collision source step (6). DOM_v2 merges all five into a single function in step3_uncollided.py. This is safe because none of these sub-steps contains a feedback loop - they are strictly sequential Beer-Lambert sweeps and linear sums. The caller receives Q along with the flux profiles, so the internal sub-step structure is completely hidden.

2. Spatial discretization is configuration, not a step

The guide lists spatial setup as Step 2, implying it is an algorithmic stage. In DOM_v2, K, dL, and G are parameters in config.py that are simply passed into each step function as arguments. This is a software engineering choice: parameters belong to configuration, not to the algorithm itself.

3. Phase tables are separated from quadrature

The guide bundles angular discretization with phase-table computation inside Step 1. DOM_v2 separates them into step1 (quadrature only) and step2 (phase tables). This makes it easy to swap quadrature rules without touching the phase function code. It also makes the most expensive setup operation (G_qq has N^2*M^2 entries) clearly visible as its own module, and it is easy to cache or profile.

4. Multiple quadrature options

The guide specifies Gauss-Legendre quadrature only. DOM_v2 step1_quadrature.py supports gauss_legendre, double_gauss (separate GL sets for each hemisphere), gauss_lobatto, and uniform. All downstream code is unaffected by this choice because only mu, w_mu, phi, w_phi are passed forward.

5. Iteration loop is internal to step4

Guide Steps 7-10 are four separate items, with Step 10 as the wrapper loop. In DOM_v2 the DD coefficient precomputation, both sweeps, the ground BC update, the scatter-source update, and the convergence check are all encapsulated inside solve_collided(). The caller simply receives the converged IC field. This hides the iterative complexity behind a clean functional interface.

6. Collided ground BC must use the current downward sweep

Both the guide and DOM_v2 stress that the collided Lambertian ground BC must be recomputed from the downward sweep of the CURRENT iteration, not from the previous one. Using a stale BC introduces a one-iteration lag that can slow convergence or, in optically thick canopies, cause oscillation. DOM_v2 makes this explicit in the code structure of step4_collided.py.

7. BRF/HDRF post-processing (extension beyond the guide)

The guide ends at energy balance. DOM_v2 adds step6_brf.py to compute the Bi-directional Reflectance Factor or HDRF at any off-nadir view angle without re-running the iteration. The method builds the view-ray source J_v from the converged scatter field and runs one additional upward DD sweep. BRF = pi * I_v(top) / (cos(theta_solar) * F_in). This extension is critical for comparing model output against satellite sensors.

8. 1-based spatial indexing

DOM_v2 uses 1-based layer indices throughout (index 0 is allocated but unused), matching the mathematical notation in the guide where k=1 is the canopy top. While non-idiomatic in Python, this makes direct comparison between code and course-note equations straightforward without an off-by-one mental translation.

Part 5 - Summary

DOM_v2 is a faithful, self-contained implementation of Ranga's 11-step Discrete Ordinates guide. Every equation in the guide has a direct counterpart in the Python code. The reduction from 11 steps to 6 files results from three decisions:

- **Algorithmic consolidation** - the five uncollided sub-steps (3a, 3b, 4, 5, 6) have no feedback between them and are merged into step3_uncollided.py.
- **Software engineering** - configuration parameters (K, dL, G) live in config.py rather than constituting a numbered algorithm step.
- **Extension** - step6_brf.py adds BRF/HDRF computation not present in the guide, exploiting the already-converged source field to avoid re-iteration.

The net result is 6 Python modules with clean functional interfaces: each function takes physical inputs and returns physical outputs, with no global state and no hidden dependencies between steps. The 11-step structure of the guide maps unambiguously onto the 6-file structure of DOM_v2 as shown in Table 1.